

Comprehensive React Deployment Guide for cPanel

1. The Problem: Why 404 Errors Occur

In a React SPA, routes like `/about-us` are managed by client-side JavaScript. When a user refreshes the page, the browser sends a direct request to the cPanel server for a physical file path. Because React apps only consist of one physical file (`index.html`) and static bundles, the server cannot find a physical file named `about-us`, triggering a 404 error.

2. The cPanel Directory Structure

When you deploy your project, your `public_html` folder should look like this. Understanding this structure is essential for professional deployment:

```
public_html/
├── .htaccess (Redirects 404 requests to index.html)
├── index.html (The app entry shell)
├── static/
│   ├── css/ (Contains main.[hash].css - all styles)
│   └── js/ (Contains main.[hash].js - all application logic)
```

3. The .htaccess Solution

To resolve the 404 issue, the `.htaccess` file acts as a traffic controller, ensuring all non-existent path requests are served by `index.html`.

```
<IfModule mod_rewrite.c>
  RewriteEngine On
  RewriteBase /
```

```
RewriteRule ^index\.html$ - [L]
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteCond %{REQUEST_FILENAME} !-l
RewriteRule . /index.html [L]
</IfModule>
```

4. Deployment Workflow

- **Development:** Write components in `.jsx` using React/MUI.
- **Build:** Run `npm run build`. This process bundles hundreds of files into optimized `.js` and `.css` files.
- **Upload:** Move the **contents** of your `build/` directory into `public_html`.

5. Navigation Dynamics

Client-Side: Clicking links (e.g., `<Link to="/">`) is handled by React Router in the browser; the server is not involved.

Server-Side: Refreshing the browser or entering a URL directly forces the browser to request the path from the server, making the `.htaccess` file mandatory.